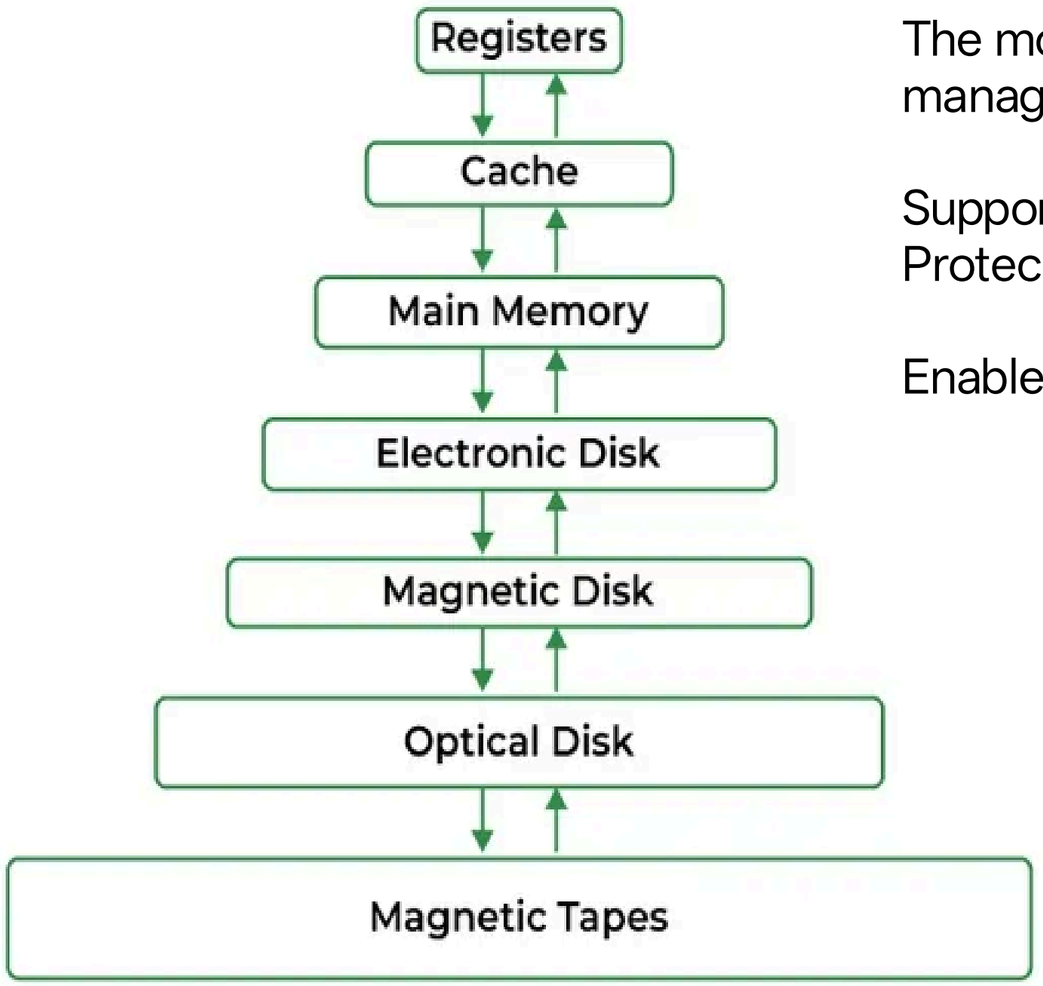
Unit-III
Memory management

By :- Dr. Manojeet Roy

Memory Management in Operating System

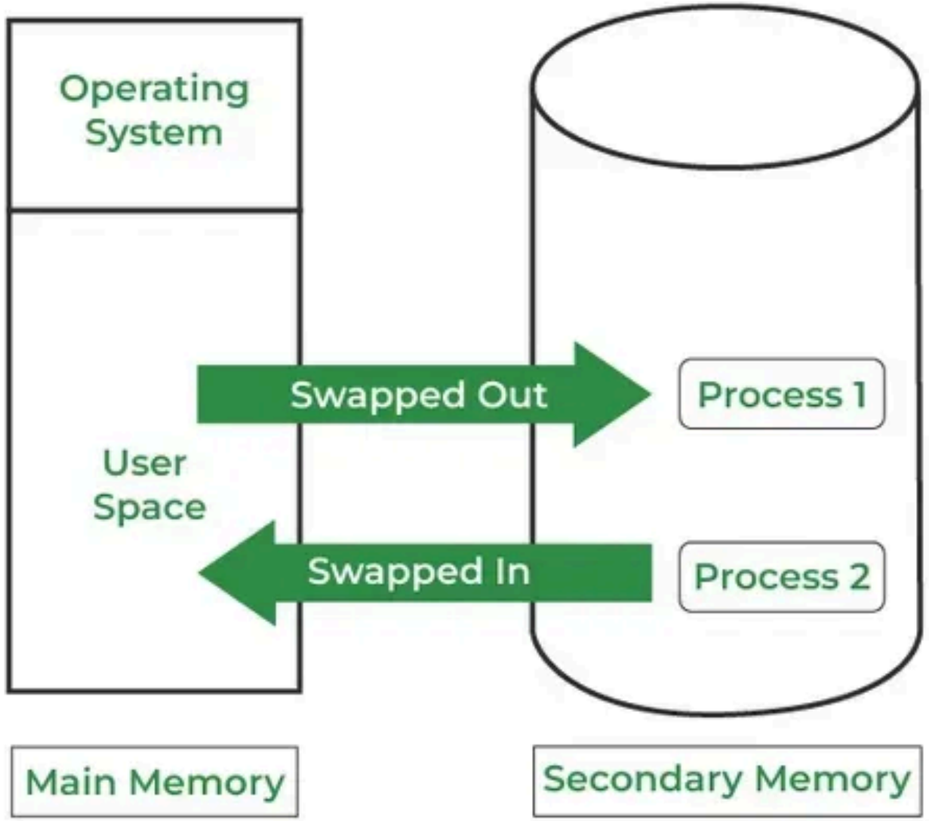


The most important function of an operating system is to manage primary memory.

Supports multiple processes simultaneously in memory.
Protects processes from unauthorised access.

Enables swapping and virtual memory efficiently.

Swapping in Operating System



This allows the operating system to manage limited RAM effectively and run multiple processes concurrently in a multiprogramming environment.

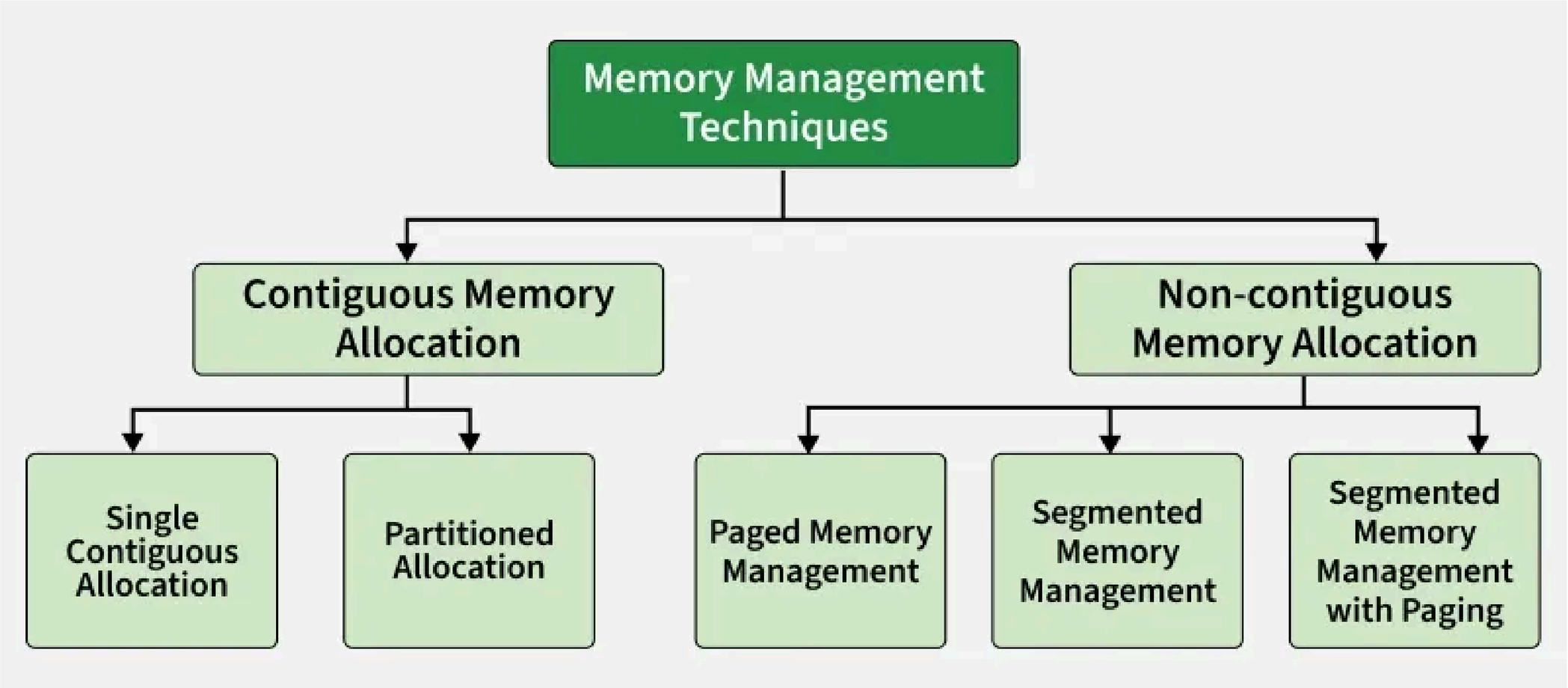
Reduces memory fragmentation by moving processes in and out of RAM. Allows large processes to run even if they exceed available RAM.

Frees RAM temporarily by suspending inactive or waiting processes. Supports virtual memory to give processes more memory than physically available.

How it work..

- A process is moved into RAM from disk when it is ready to execute.
- After the process has run for a while (or if higher-priority processes need memory), it may be temporarily swapped out to disk.
- The CPU scheduler decides which process to swap in and out, often based on priority or scheduling policies.
- When the swapped-out process is needed again, it is swapped back into memory to continue execution.

Techniques in Memory Allocation



Cont...

Fixed Partition Allocation

- Memory is divided into a fixed number of partitions
- Each partition has a fixed size
- Each partition can store only one process
- Leads to **internal fragmentation**
- Once partitions are defined operating system keeps track of the status of memory partitions it is done through a data structure called a partition table.

Cont...

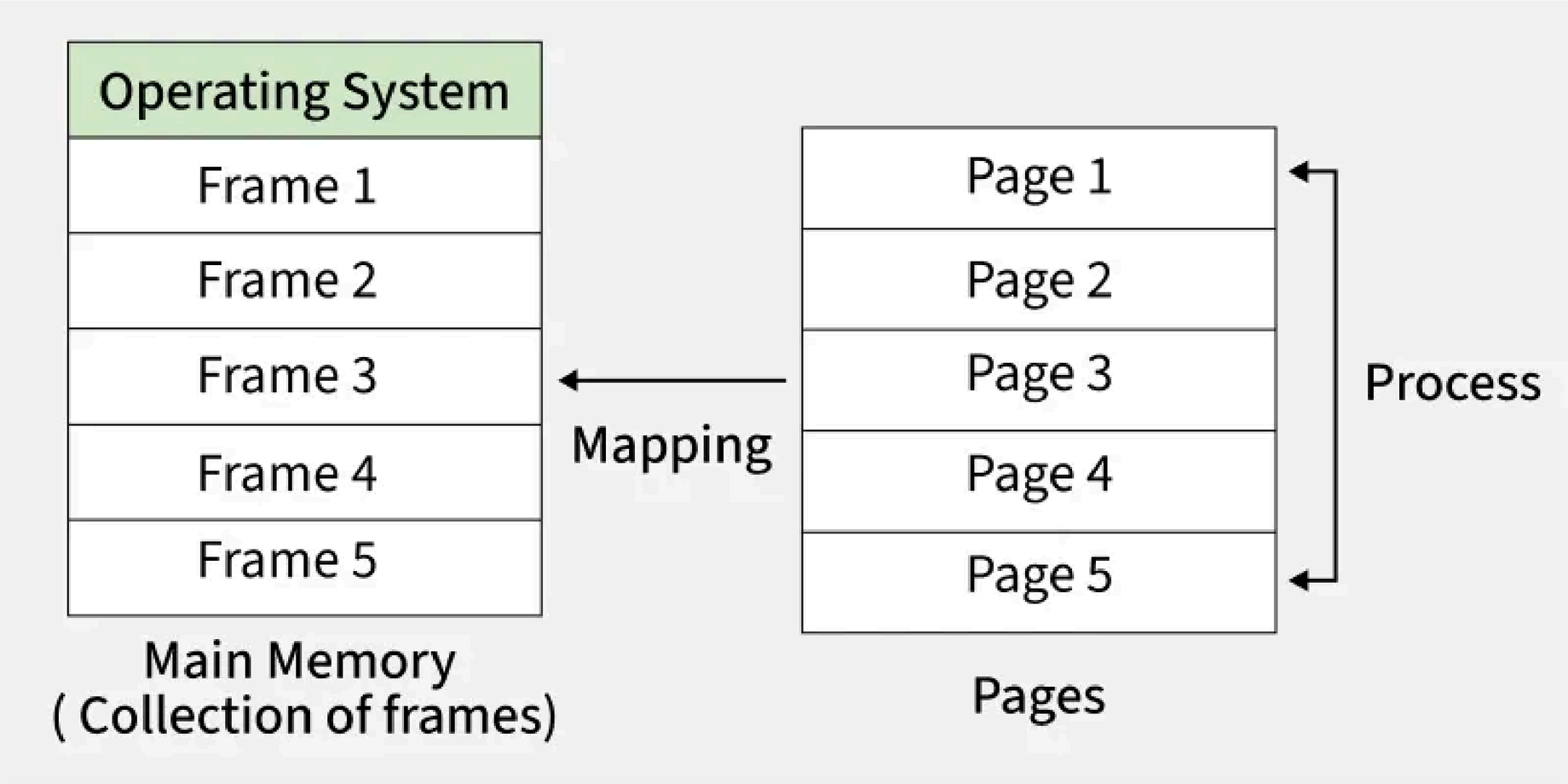
Features of Non-Contiguous Memory Allocation

- A process can be stored in multiple memory locations
- Improves utilization of available memory
- Reduces **external fragmentation**
- Requires address translation using hardware support (MMU)

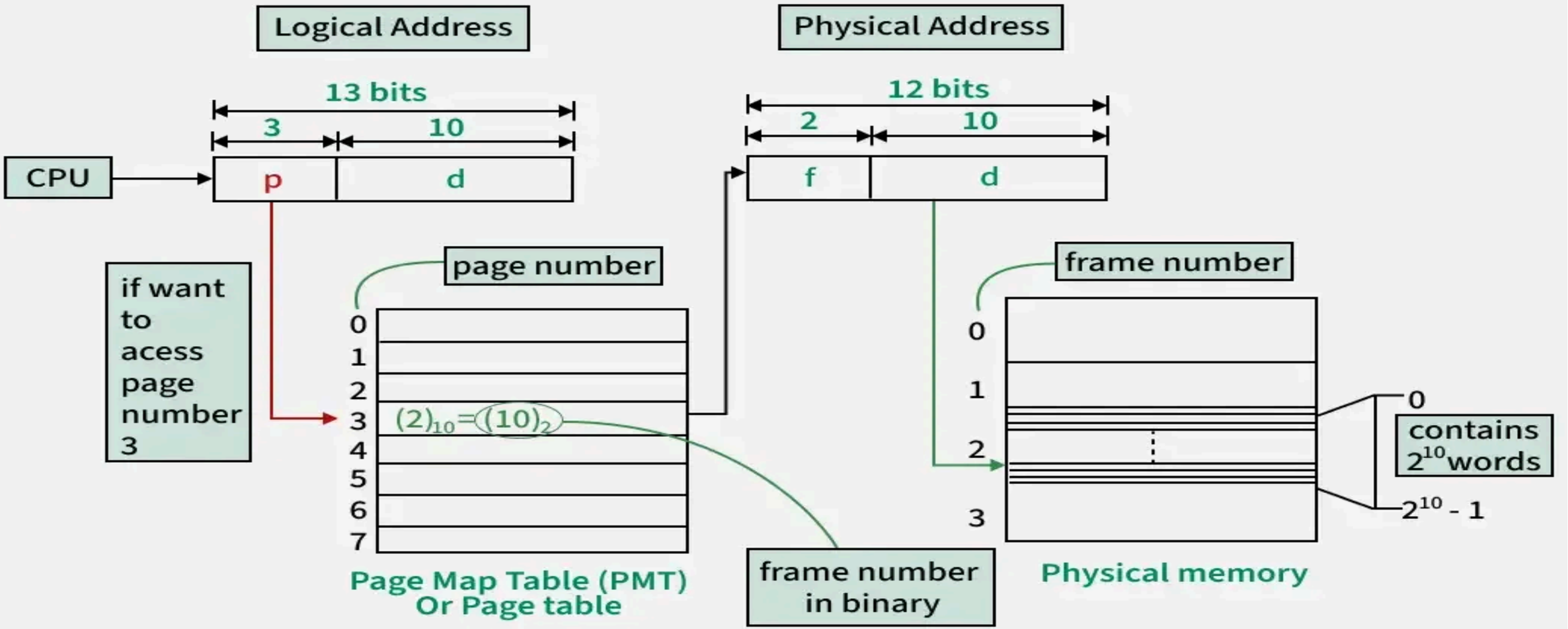
Techniques Used in Non-Contiguous Memory Allocation

- **Paging**: Divides a process into fixed-size pages and memory into frames of the same size
- **Segmentation:** Divides a process into logical segments of variable size such as code, data, and stack
- **Segmentation with Paging:** Combines logical segmentation with paging to reduce fragmentation

Paging


Operating System
Frame 1
Frame 2
Frame 3
Frame 4
Frame 5
Main Memory
(Collection of frames)
Mapping
Page 1
Page 2
Page 3
Page 4
Page 5
Pages
Process

Working of Paging



1.What happens immediately after a 'TLB miss' occurs in this translation process?
2. In the provided diagram, what is the primary purpose of the 'd' component in the Logical Address?

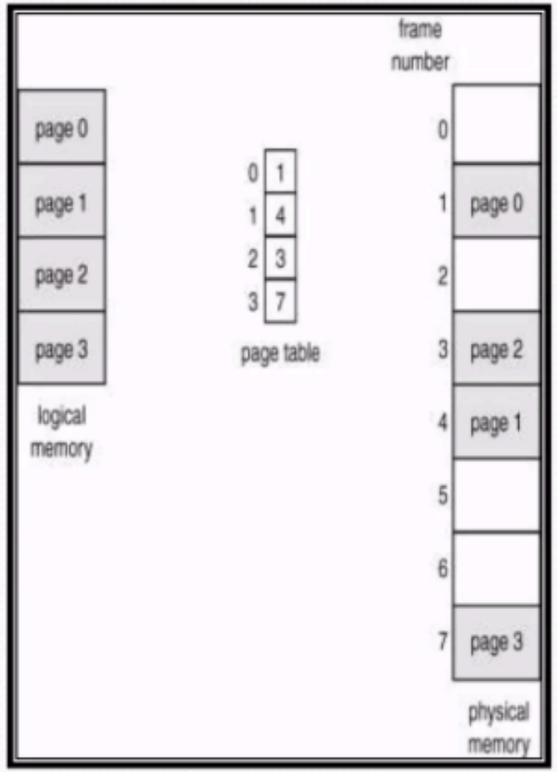
3. Based on the TLB table in the diagram, what frame number corresponds to Page Number 2?
4. Why is a TLB hit faster than a TLB miss?
5.In the Page Map Table shown, which page number is currently being accessed during the 'TLB miss' path?
6. Which component is responsible for generating the 'Logical Address’ ?
7. If a page is not found in the Page Map Table, what event usually occurs?
page fault OR page miss OR Data Deletion
8. What is the relationship between the number of entries in the TLB and the Page Map Table?
9. Which part of the final Physical Address is derived directly from the TLB or Page Table lookup?
10. In the 'Physical Address' box on the right, what do the numbers 0, 1, 2, and 3 represent?

Steps Involved in Paging :

- **Step 1 Divide Memory :** Logical -> Pages, Physical -> Frames .
- **Step 2 Allocate Pages :** Load pages into available frames.
- **Step 3 Page Table :** Map logical pages to physical frames.
- **Step 4 Translate Address :** Convert logical to physical address.
- **Step 5 Handle Page Fault :** Load missing pages from disk.
- **Step 6 Run Program :** CPU uses page table during execution.

Logical Structure of Page Table

A **page table** is the data structure used by a virtual memory system in a computer operating system to store the mapping between virtual addresses and physical addresses



Cont..

Paging Hardware With TLB

- Associative memory – parallel search

Page #	Frame #

- Address translation (A′, A′′)
  - If A′ is in associative register, get frame # out.
  - Otherwise get frame # from page table in memory

Common Techniques used for structuring the page table are :

- Hierarchical paging
- Hashed page tables
- Inverted page tables

HIERARCHICAL PAGING:

TWO LEVEL PAGING:

- A logical address (on 32-bit machine with 4k page size) is divided into:

  A page number consisting of 20 bits.

  A page offset consisting of 12 bits.

- Since the page table is paged , the page number is further divided into:

  A 10-bit page number.

  A 10-bit page offset.

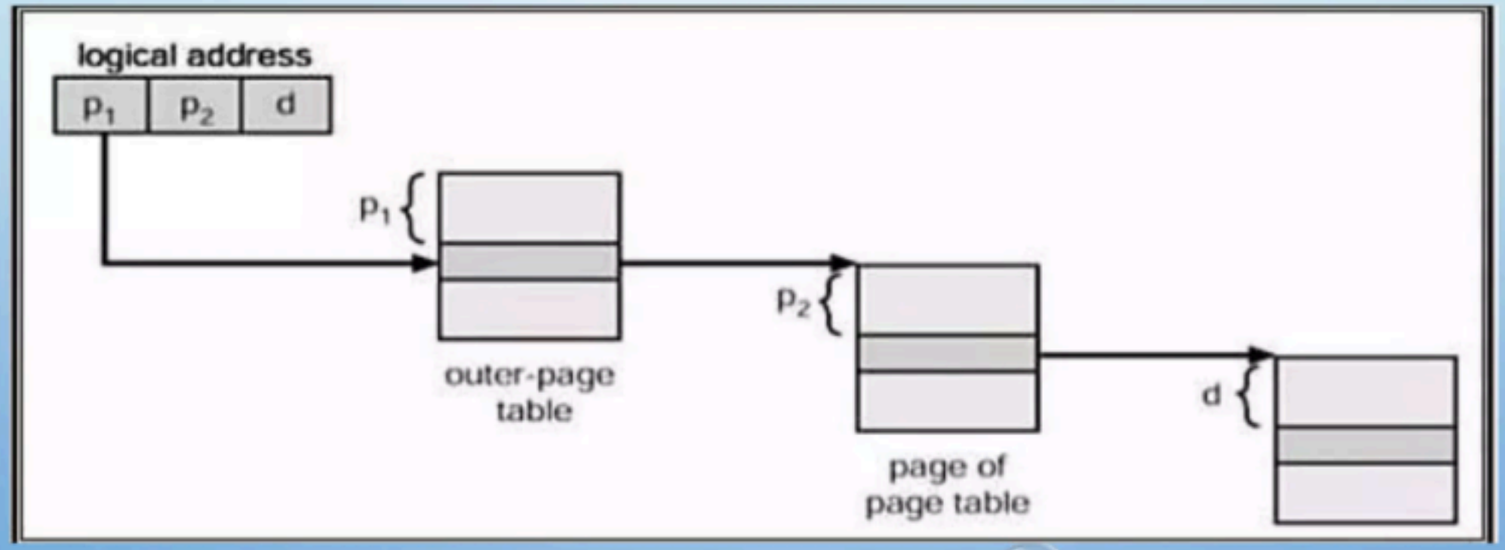
- Thus a logical address is as follows:

pi	p2	d

page number | page offset

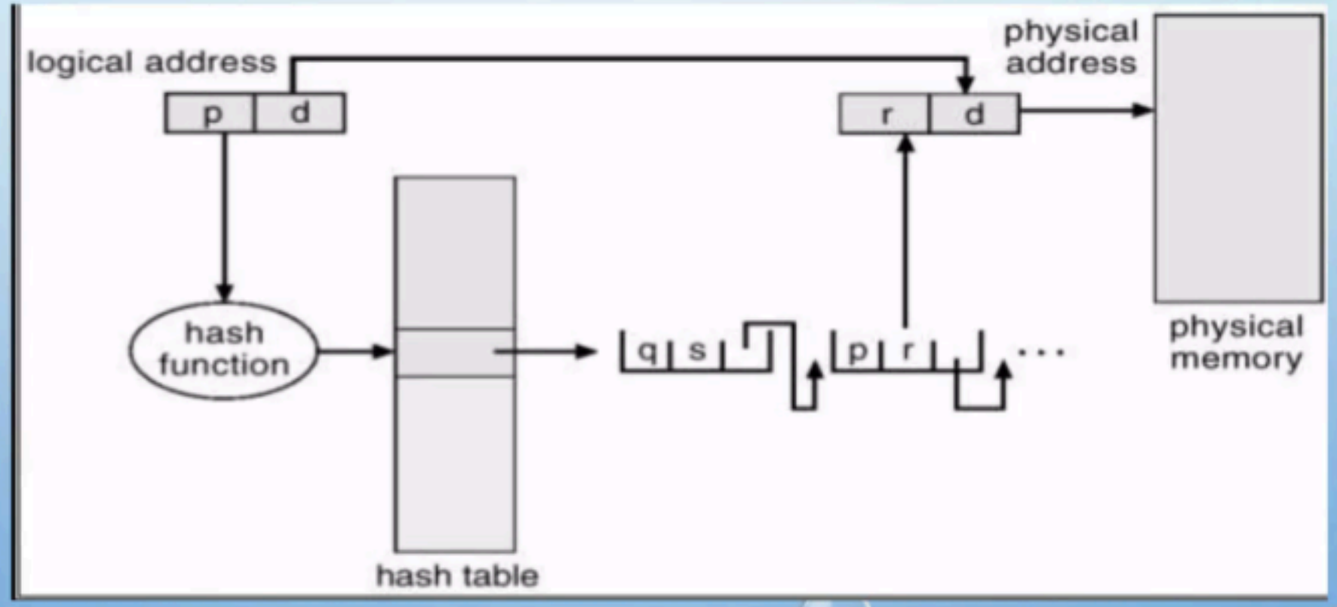
Cont..

- Address translation scheme for the two level paging is as follows:



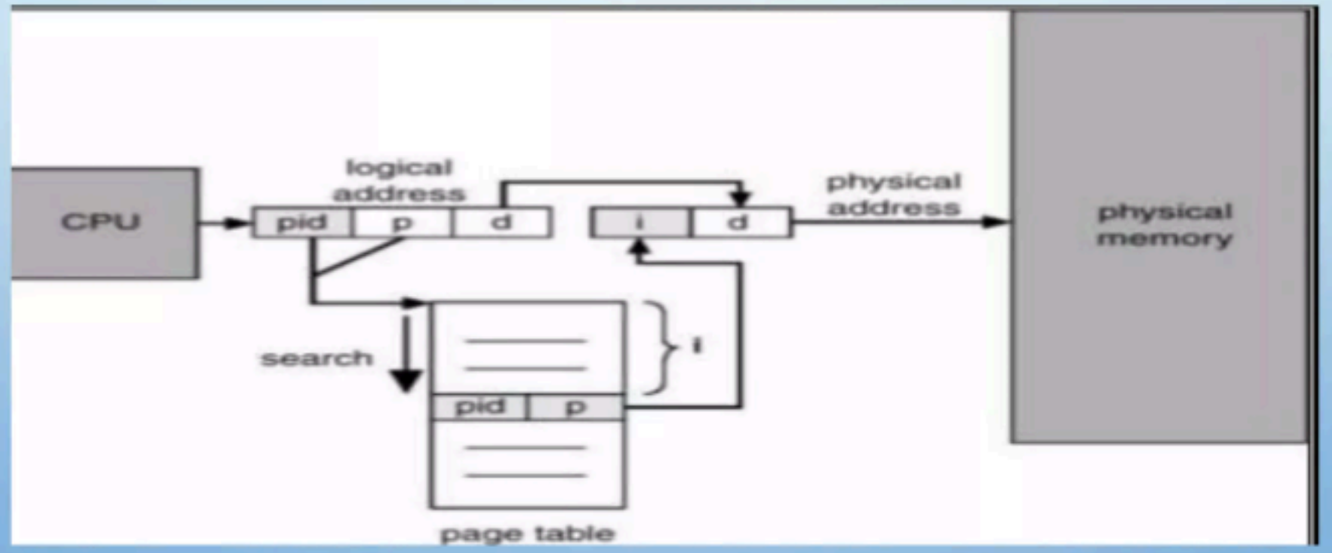
Hashed paging

Address translation scheme for the hashed page table is as follows:



Cont..

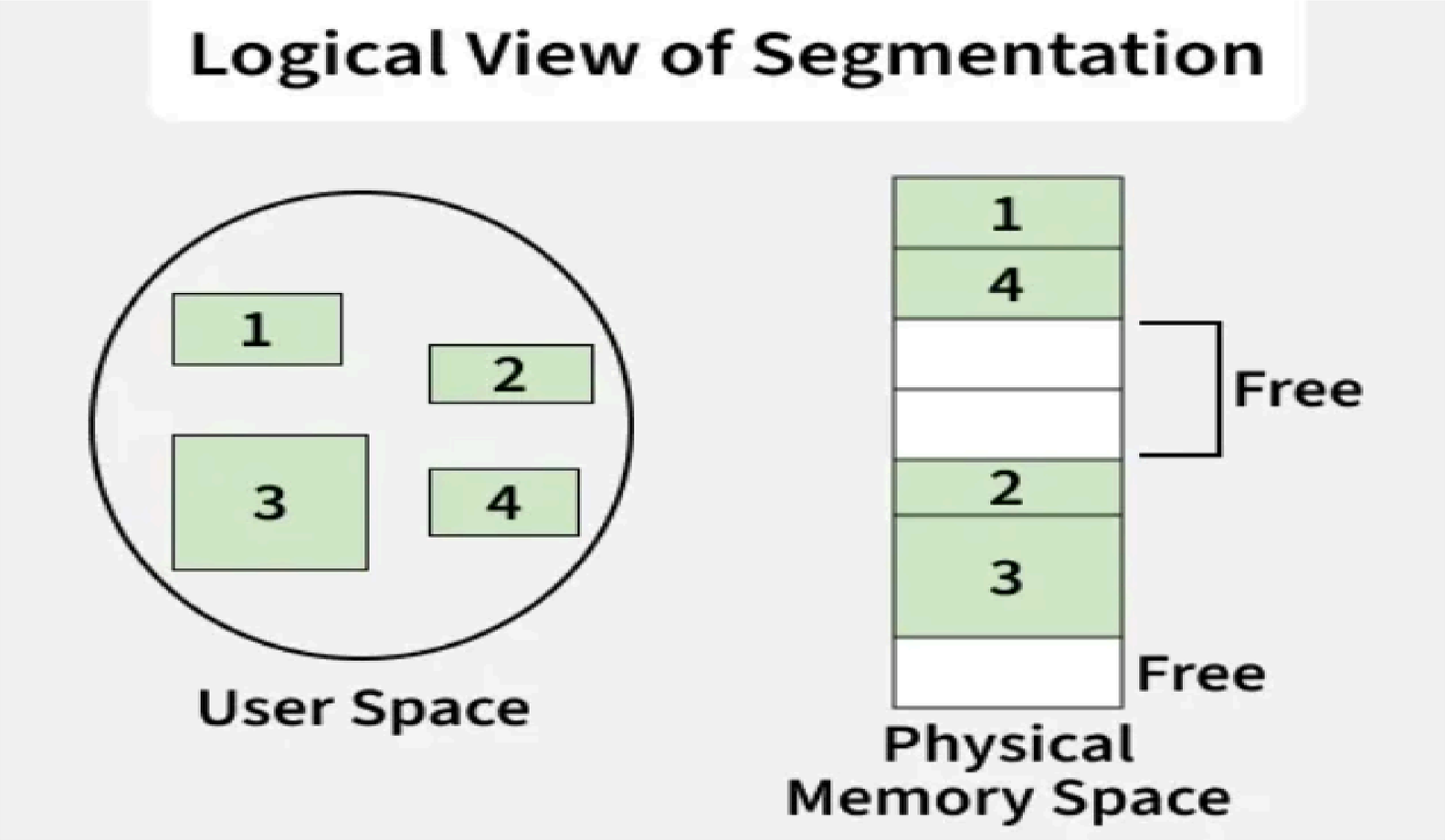
Address translation scheme for the Inverted page table is as follows:



Segmentation

Segmentation is a memory management technique where a process is divided into variable-sized chunks called **segments**. Unlike paging, segmentation matches the user's logical view of a program (functions, arrays, modules) to physical memory.

Segmentation



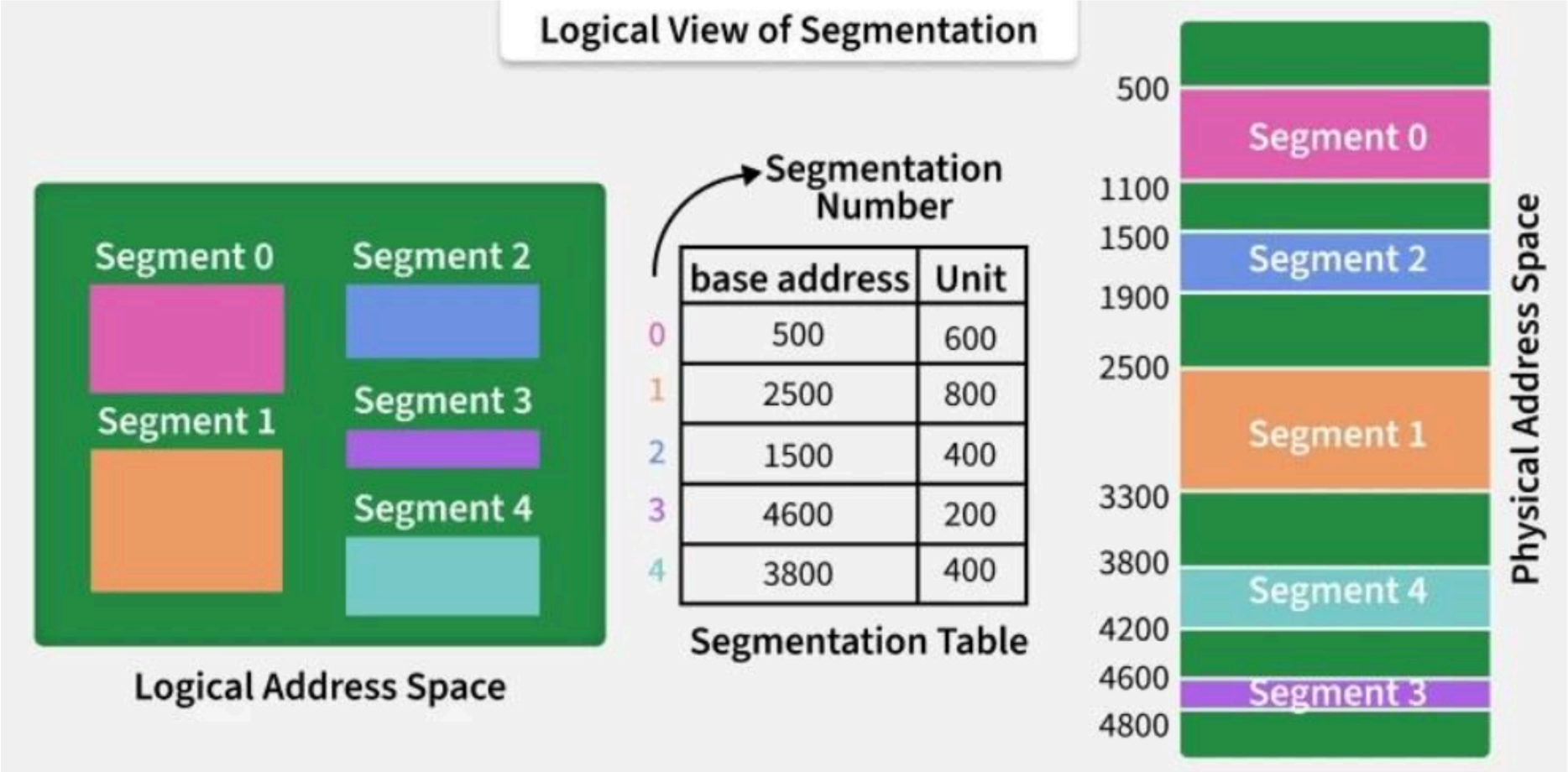
Key Features of Segmentation

1. Variable-sized divisions: Segments can have different lengths, depending on the program's requirements.
2. Logical division of memory: Segments represent meaningful units like code, stack, data, or modules.
3. Two-part address: A logical address consists of:

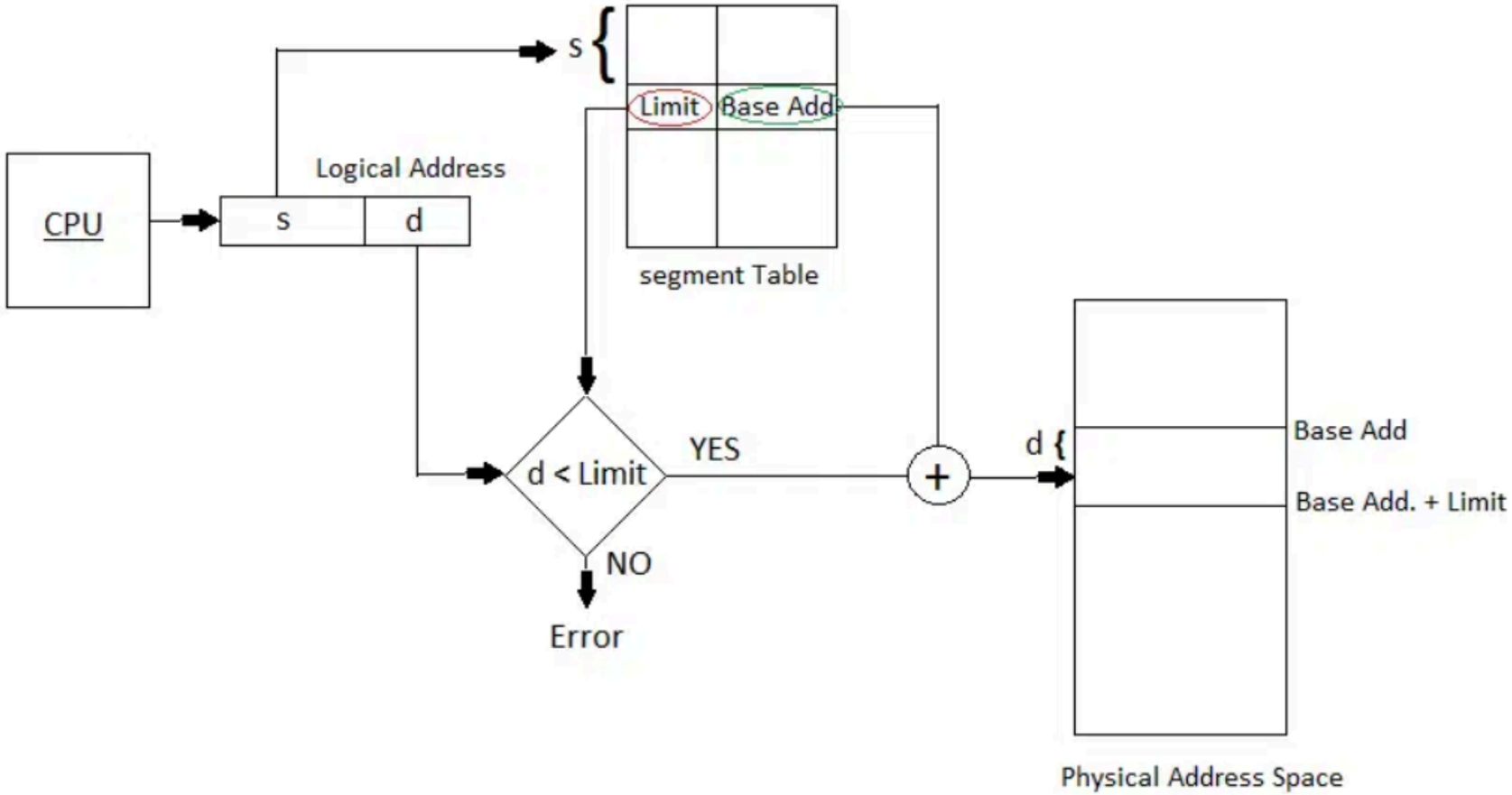
Segment number (s): Identifies which segment is being referred to.

Offset (d): Specifies the exact location within that segment.
Logical Address = ⟨Segment number, Offset⟩

Logical View



Cont..


s
Limit
Base Add
segment Table
CPU
Logical Address
s
d
d < Limit
YES
NO
Error
+
d
Base Add
Base Add. + Limit
Physical Address Space